3C

DESARROLLO DE SOFTWARE

Desarrollo de aplicaciones de Software para la solución de problemas

Saber, valorar, elegir

PADRE ARRUPE
Colegio Español

DOCENTE: Daniel Sosa

COLEGIO ESPAÑOL PADRE ARRUPE

Colegio internacional.

Tercer Año de Bachillerato de Software

Desarrollo de aplicaciones de Software para la solución de problemas

Periodo 1.

GUÍA 3 – Introducción a GUI en Java

OBJETIVOS DE LA PRÁCTICA

Objetivo: Comprender la estructura y sintaxis de Java, manejar variables y tipos de datos, utilizar operadores y aplicar estructuras de control de flujo.

INDICACIÓN GENERAL: Introducir variables y tipos de datos, explicar operadores y expresiones, y enseñar estructuras de control de flujo

Nota: La guía forma parte integral de la clase, por lo que su finalización se revisa durante la misma.

INTRODUCCIÓN TEÓRICA

1. **Interfaces Gráficas (GUI)**:

Las interfaces gráficas permiten a los desarrolladores crear aplicaciones con una interfaz visual, lo que facilita la interacción del usuario con el software.
NetBeans ofrece un entorno de desarrollo integrado (IDE) que facilita la creación de interfaces gráficas en Java mediante su herramienta de diseño GUI Builder.
Los componentes GUI en NetBeans incluyen botones, cajas de texto, etiquetas, paneles y otros elementos que pueden ser colocados y configurados visualmente en la interfaz de usuario.
El GUI Builder de NetBeans permite la creación de interfaces gráficas mediante un enfoque de arrastrar y soltar, lo que simplifica el proceso de diseño y desarrollo.

2. **JDBC (Java Database Connectivity)**:

JDBC es una API de Java que proporciona métodos y clases para interactuar con bases de datos relacionales desde aplicaciones Java.
JDBC facilita la conexión, consulta, actualización y manipulación de datos en bases de datos SQL como MySQL, PostgreSQL, Oracle, etc.
NetBeans ofrece soporte completo para JDBC, lo que significa que puedes integrar fácilmente la conectividad a bases de datos en tus aplicaciones Java.
Con NetBeans, puedes crear conexiones JDBC, ejecutar consultas SQL, recuperar resultados y manejar transacciones desde tu aplicación Java de manera visual y eficiente.

3. **Integración en NetBeans**:

NetBeans proporciona herramientas visuales para trabajar con JDBC, como asistentes para crear conexiones a bases de datos, consultas SQL y manejo de resultados.
La integración de interfaces gráficas y JDBC en NetBeans permite a los desarrolladores crear aplicaciones completas que interactúan con bases de datos de manera intuitiva y eficaz.
Al aprovechar las capacidades de diseño de interfaces gráficas y la potencia de JDBC en NetBeans, los desarrolladores pueden crear aplicaciones robustas y funcionales con una experiencia de usuario mejorada.
En resumen, NetBeans ofrece un entorno de desarrollo completo para la creación de aplicaciones Java que incluyen interfaces gráficas y conectividad a bases de datos mediante JDBC. Esta integración simplifica el proceso de desarrollo y permite a los desarrolladores crear aplicaciones visuales y potentes de manera eficiente.

REALIZACIÓN DE PRÁCTICA

1. Crear un nuevo proyecto con el nombre de **“guijava”**
2. Para poder realizar una conexión crearemos una clase que tenga los métodos necesarios y que se llame **“Conexion.java”**, agregar el siguiente **código:**

```java
package guijava2;

import java.sql.*;

public class Conexion {

    private Connection conexion = null;
    private Statement s = null;
    private ResultSet rs = null;
    private String query = "";

    // Constructor
    public Conexion() throws SQLException {
        try {
            // Obtenemos el driver para MySQL
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Se obtiene una conexión con la base de datos
            conexion =
DriverManager.getConnection("jdbc:mysql://localhost/guijava", "root",
"");
            // Permite ejecutar sentencias SQL sin parámetros
            s = conexion.createStatement();
        } catch (ClassNotFoundException e1) {
            // Error si no se puede leer el driver de MySQL
            System.out.println("ERROR: No se encuentra el driver de la
BD: " + e1.getMessage());
        }
    }

    // Método que permite obtener los valores del ResultSet
    public ResultSet getRs() {
        return rs;
    }

    // Método que permite fijar la tabla resultado de la consulta SQL
realizada
    public void setRs(String consulta) {
        try {
            // Creamos el Statement con el tipo de ResultSet deseado
            s =
conexion.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
            this.rs = s.executeQuery(consulta);
        } catch (SQLException e2) {
            System.out.println("ERROR: Fallo en SQL: " +
e2.getMessage());
        }
    }
```

```
    // Método que recibe un SQL como parámetro que sea un update, insert,
o delete
    public void setQuery(String query) throws SQLException {
        this.s.executeUpdate(query);
    }

    // Método que cierra la conexión
    public void cerrarConexion() throws SQLException {
        conexion.close();
    }

    ResultSet getRs(String string) {
        try {
        // Creamos el Statement con el tipo de ResultSet deseado
        Statement statement =
conexion.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
        return statement.executeQuery(string);
    } catch (SQLException e) {
        System.out.println("ERROR: Fallo en SQL: " + e.getMessage());
        return null; // o lanza una excepción, dependiendo de tu manejo
de errores
    }
    }
}
```

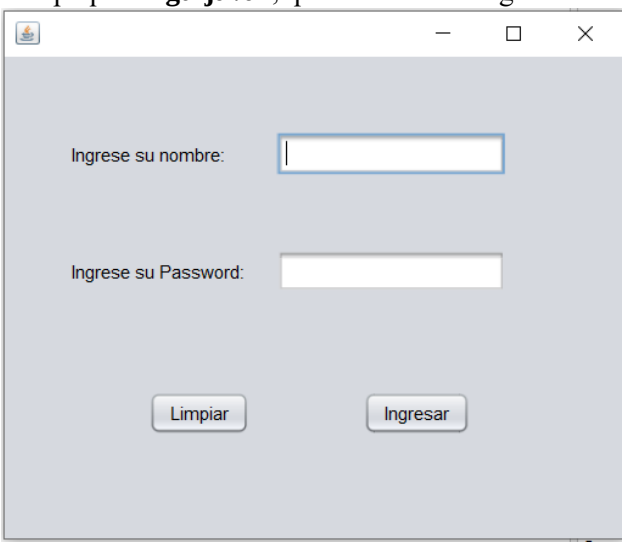
Nota: No olvide agregar el driver MySQL JDBC Driver al proyecto (recomendado) o a la dirección respectiva.

BASE DE DATOS: https://drive.google.com/file/d/1xntaenoVPeLsTUWDNyDR9RwAS-P8aHsa/view?usp=sharing (importar desde PHPMyAdmin)

DEBE CREAR EL PAQUETE sv.org.arrupe.util dentro de el creara la clase CheckPassword y digitara el siguiente código.

```java
package sv.org.arrupe.util;

/**
 *
 * @author Daniel Sosa
 */
public class CheckPassword {
    public boolean verificarPassword(char passArray[]){
        for(int i=0;i<passArray.length;i++){
            char c = passArray[i];
            //Si no es una letra no es valido
            if(!Character.isLetterOrDigit(ch: c)){
                return false;
            }
        }
        return true;
    }
}
```

3. Nuestro propósito para esta guía será crear un formulario de ingreso que permita validar un usuario que sea de tipo **"admin"** si esto es verdadero pasaremos a un frame de tipo Mdi que permitirá abrir un mantenimiento básico, para ello crear un JframeForm llamado **Logueo** dentro del paquete "**guijava**", que se vea de la siguiente manera.



No olvide importar los siguientes paquetes dentro del código del Formulario

```
package guijava2;
import java.util.logging.Level;
import java.util.logging.Logger;
import java.sql.*;
import javax.swing.JOptionPane;
import sv.org.arrupe.util.CheckPassword;
/**
 *
 * @author Daniel Sosa
 */
public class Logueo extends javax.swing.JFrame {
```

4. Cambiar las propiedades de cada componte según como se muestra en la siguiente tabla.

Control	Propiedad	Evento	Código	Valor
jTextField1	name		Variable name	txtNombre
jPasswordField1	name		Variable name	txtPassword
jButton1	text	actionPerformed		Limpiar
jButton2	text	actionPerformed		Ingresar

5. Modificar el constructor de este Jframe para que quede de la siguiente forma, esto para que la ventana aparezca posicionada en el centro.

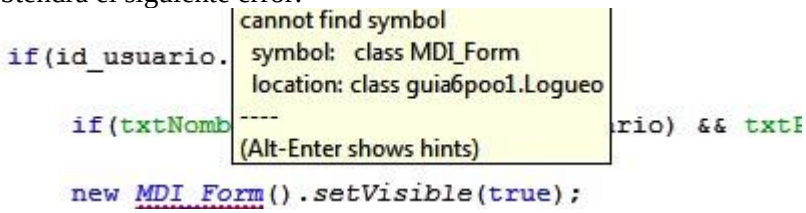
```
public Logueo() {
    initComponents();
    setLocationRelativeTo(null);
}
```

6. Ahora utilizamos el evento **actionPerformed** del botón ingresar, dentro de él digitar el siguiente código:

```
private void Ingresar(java.awt.event.ActionEvent evt)
{
    // TODO add your handling code here:
    String id_tipo = "";
String usuario = "";
String password = "";
try {
    CheckPassword verificar = new CheckPassword();
    char passArray[] = txtPassword.getPassword();
    if (verificar.verificarPassword(passArray)) {
        String pass = new String(passArray);
        Conexion con = new Conexion();//creamos el objeto para la
conexion
        con.setRs("select tipo_usuario_id,nombre_usuario,password from
usuarios where nombre_usuario='" + txtNombre.getText() + "'");//consulta
```

```
        ResultSet valor = con.getRs();//obtenemos los valores
        if (valor.next()) {// Verificamos si hay filas devueltas
            id_tipo = valor.getString(1);//obtenemos el id del tipo de
usuario
            usuario = valor.getString(2);//obtenemos el usuario
            password = valor.getString(3);//obtenemos el password del
usuario
            System.out.println("Password: " + pass + " " + password);
            System.out.println("Usuario: " + txtNombre.getText() + " " +
usuario);//verificamos si el usuario y el password de la base son iguales
a los ingresados en los txt
            if (txtNombre.getText().equals(usuario) &&
pass.equals(password)) {//si este es un usuario de tipo administrador
                System.out.println("Id: " + id_tipo);
                if (id_tipo.equals("1")) {
                    new MDI_Form().setVisible(true);
                    this.dispose();
                } else {
                    JOptionPane.showMessageDialog(this, "Empleado
autorizado");
                }
            } else {
                JOptionPane.showMessageDialog(this, "Usuario ó Password
incorrecto");
            }
        } else {
            JOptionPane.showMessageDialog(this, "Usuario no encontrado");
        }
    } else {
    }
} catch (SQLException ex) {
    Logger.getLogger(Logueo.class.getName()).log(Level.SEVERE, null, ex);
}
}
```

Tome en cuenta que obtendrá el siguiente error:

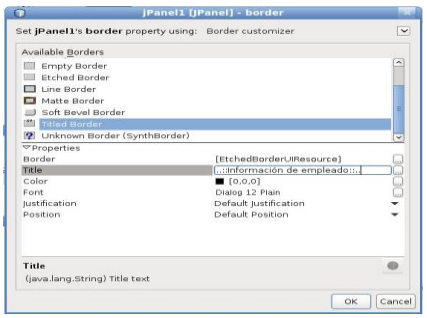


Esto es debido a que el formulario que se llamará MDI_Form será creado en el paso 11 de esta guía.

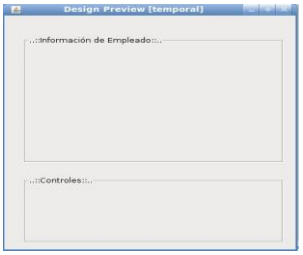
7. Para darle funcionalidad al botón Limpiar seleccionar el evento **actionPerformed** y agregar el siguiente código:

```
private void Limpiar(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    txtNombre.setText(t: "");
    txtPassword.setText(t: "");
}
```

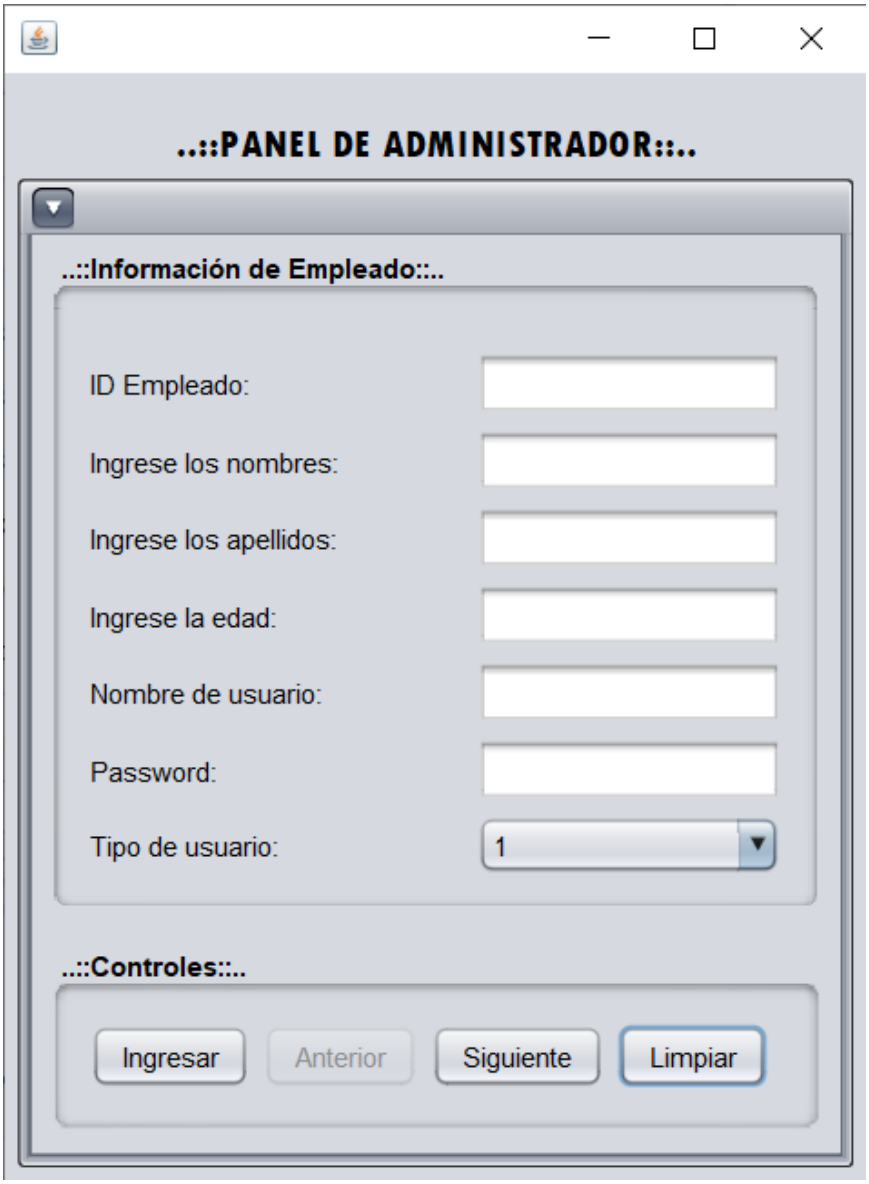
8. Finalizado la creación del formulario de ingreso de usuario, pasaremos a crear un formulario
   Mdi dentro del paquete "**guijava**" llamado **"MDI_Form"** y agregar un Jframe interno (JInternalFrameForm) el cual se llamara **"MantenimientoEmpleados"** dentro del mismo paquete.
9. Agregar 2 Panel al FrameMantenimientoEmpleados, ahora seleccionar uno de los controlespanel ir a las propiedades y seleccionar **"border"**, aparecerá una pantalla como la siguiente



Ya en la ventana seleccionar **"TitledBorder"** y modificar la propiedad title como se ve en la imagen, modificar el segundo **Panel** para que al final se vea de la siguiente manera.



10. Modificar el Frame interno para que se vea de la siguiente manera.



11. Cambiar las propiedades de cada componte según como se muestra en la siguiente tabla.

Control	Propiedad	Eventos	Código	Valor
JInternalFrame	title			Ingreso Persona
jLabel1	text			ID:
			Variable Name	lblId
jLabel2	text			Ingrese los nombres:
			Variable Name	lblNombres
jLabel3	text			Ingrese los apellidos:
			Variable Name	lblApellidos
jLabel4	text			Ingrese la edad:
			Variable Name	lblEdad
jLabel5	text			Nombre de usuario

			Variable Name	lblNombreUsuario
jLabel6	text			Password:
			Variable Name	lblPasswordUsuario
jLabel7	text			Tipo de Usuario:
			Variable Name	lblTipoUsuario
jTextField1	name			txtID
jTextField2	name			txtNombres
jTextField3	name			txtApellidos
jTextField4	name			txtEdad
jTextField5	name			txtNombreUsuario
jPasswordField1	name			txtPassword
jComboBox1	name			cmbTipoUsuario
jButton1	name	actionPerformed		btnIngresar
	text			Ingresar
jButton2	name	actionPerformed		btnAnterior
	text			Anterior
jButton3	name	actionPerformed		btnSiguiente
	text			Siguiente
jButton4	name	actionPerformed		btnLimpiar
	text			Limpiar

12. Importar antes de la **public class** los siguiente paquetes luego, dentro de la clase, agregar las siguientes propiedades e inicializar el constructor tal y como se muestra en la siguiente imagen.

```java
package guijava2;
import java.awt.Toolkit;
import java.sql.*;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.JOptionPane;
import sv.org.arrupe.util.CheckPassword;
/**
 *
 * @author Daniel Sosa
 */
public class MDI_Form extends javax.swing.JFrame {

    /**
     * Creates new form MDI_Form
     */
    ResultSet empleados;
    ResultSet llenarcombo;
    static int bandera=0;
    public MDI_Form() throws SQLException {
        initComponents();
        iniciarValores();
    }
```

Nota:iniciarValores() es un método que va a crear en el siguiente paso, no tome en cuenta el error de "*método no definido*".

13. Como se puede observar en el constructor aparte de que inicializar los componentes, también llama a un método creado por nosotros.Usted debe crearlo y codificarlo tal y como se ve en la imagen:

```java
public void iniciarValores() throws SQLException{
    Conexion con = new Conexion();//he movido esta linea de codigo
    con.setRs(consulta: "SELECT tipo_usuario_id, nombres, apellidos, edad,nombre_usuario,password FROM usuarios;");

    empleados = (ResultSet) con.getRs();
    empleados.last();
    empleados.beforeFirst();
    empleados.next();

    Conexion con2 = new Conexion();

    con2.setRs(consulta: "select tipo_usuario_id from usuarios");
    cmbTipoUsuario.removeAllItems();
    llenarcombo=(ResultSet) con2.getRs();
    while(llenarcombo.next()){
        cmbTipoUsuario.addItem(item: llenarcombo.getString(i: 1));
    }
    con2.cerrarConexion();
    llenarTxtbox();
    btnAnterior.setEnabled(b: false);
    btnSiguiente.setEnabled(b: true);
}
```

Nota:llenarTxtbox() es un método que va a crear en el paso 18, no tome en cuenta el error de "*método no definido*".

14. Ahora pasaremos a crear la funcionalidad de los botones Anterior, Siguiente y Limpiar, seleccionar el evento **actionPerformed**y los digitaremos tal y como se muestra en la imagen.

Para el botón Anterior

```
private void Anterior(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    try {
        anterior();
    } catch (SQLException ex) {
        try {
            anterior();
        } catch (SQLException ex1) {
            Logger.getLogger(name: MDI_Form.class.getName()).log(level: Level.SEVERE, msg:null, thrown: ex1);
        }
    }
}
```

Para el botón Limpiar

```
private void Limpiar(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    limpiarText();
}
```

Para el botón Siguiente

```
private void Siguiente(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    try {
        siguiente();
    } catch (SQLException ex) {
        try {
            siguiente();
        } catch (SQLException ex1) {
            Logger.getLogger(name: MDI_Form.class.getName()).log(level: Level.SEVERE, msg:null, thrown: ex1);
        }
    }
}
```

Nota: todos los métodos mostrados se crearán en el siguiente paso, no tome en cuenta el error de "*método no definido*" y los llamados de atención de SQLException no lanzado. (imagen de referencia).

```
254     private void btnAnteriorActionPerformed(java
255         // TODO add your handling code here:
256         try{
            anterior();
exception java.sql.SQLException is never thrown in body of
corresponding try statement
259 ----
260 (Alt-Enter shows hints)
            }catch(SQLException ex1){
263             Logger.getLogger(MantenimientoE
```

15. Como se puede observar cada evento llama métodos importantes que generan la funcionalidad de los botones Anterior y Siguiente, los métodos son detallados a continuación (si no comprende su funcionamiento consultar con su instructor).

```
public void siguiente() throws SQLException{
    if(empleados.isAfterLast()==false){
        btnAnterior.setEnabled(b: true);
        empleados.next();
        llenarTxtbox();
    }
    else{
        JOptionPane.showMessageDialog(parentComponent: this, message: "Ya no existen más registros para recorrer");
        empleados.previous();
        btnSiguiente.setEnabled(b: false);
    }
}
private void anterior() throws SQLException{
    if(empleados.isBeforeFirst()==false){
        btnAnterior.setEnabled(b: true);
        empleados.previous();
        llenarTxtbox();
    }
    else{
        JOptionPane.showMessageDialog(parentComponent: this, message: "Ya no existen más registros para recorrer");
        empleados.next();
        btnSiguiente.setEnabled(b: false);
    }
}
```

```
private void limpiarText(){
    txtID.setText(t: "");
    txtNombres.setText(t: "");
    txtApellidos.setText(t: "");
    txtEdad.setText(t: "");
    txtNombreUsuario.setText(t: "");
    cmbTipoUsuario.setSelectedIndex(anIndex: 0);
    txtPassword.setText(t: "");
}

private void llenarTxtbox() throws SQLException{
    txtID.setText(t: String.valueOf(i: empleados.getInt(string: "tipo_usuario_id")));
txtNombres.setText(t: empleados.getString(string: "nombres"));
txtApellidos.setText(t: empleados.getString(string: "apellidos"));
txtEdad.setText(t: String.valueOf(i: empleados.getInt(string: "edad")));
txtNombreUsuario.setText(t: empleados.getString(string: "nombre_usuario"));
txtPassword.setText(t: empleados.getString(string: "password"));

// Obtener el tipo de usuario como un String del combo
String tipoUsuario = String.valueOf(i: empleados.getInt(string: "tipo_usuario_id"));
cmbTipoUsuario.setSelectedItem(anObject: tipoUsuario);
}
```

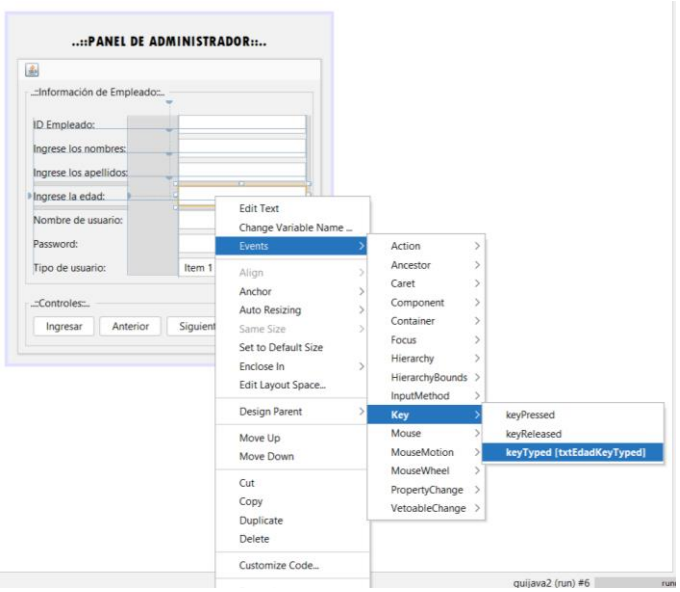
16. Ahora generaremos la funcionalidad del botón ingresar para agregar un nuevo registro a la base de datos, digite el código necesario para este paso, que utiliza el eventoactionPerformed.

```
private void Ingresar(java.awt.event.ActionEvent evt)
{
    // TODO add your handling code here:
    try {
    if (btnIngresar.getText().equals("Ingresar")) {
        btnIngresar.setText("Guardar");
        btnAnterior.setEnabled(false);
        btnSiguiente.setEnabled(false);
        limpiarText();
    } else {
        Conexion con2 = new Conexion();
        CheckPassword verificar = new CheckPassword();
        char passArray[] = txtPassword.getPassword();
        if (verificar.verificarPassword(passArray)) {
            String password = new String(txtPassword.getPassword());

            // Obtener el tipo de usuario seleccionado en el combo
            String tipoUsuario = (String)
cmbTipoUsuario.getSelectedItem();

            // Consultar la tabla tipo_usuario para obtener el
tipo_usuario_id correspondiente
            ResultSet rs = con2.getRs("SELECT tipo_usuario_id FROM
tipo_usuario WHERE tipo_usuario_id = '" + tipoUsuario + "'");

            if (rs.next()) {
                // Si se encuentra el tipo de usuario, obtener su ID
                int tipoUsuarioID = rs.getInt("tipo_usuario_id");

                // Realizar la inserción en la tabla usuarios con el tipo
de usuario ID obtenido
                con2.setQuery("INSERT INTO usuarios (usuario_id,
tipo_usuario_id, nombres, apellidos, edad, nombre_usuario, password)
VALUES ("
                                + txtID.getText() + ", " + tipoUsuarioID
+ ", '" + txtNombres.getText() + "', '" + txtApellidos.getText() + "', "
                                + Integer.parseInt(txtEdad.getText()) +
", '" + txtNombreUsuario.getText() + "', '" + password + "')");

                con2.cerrarConexion();
                btnIngresar.setText("Ingresar");
                JOptionPane.showMessageDialog(this, "Usuario Ingresado
Exitosamente");
                limpiarText();
                iniciarValores();
```

```
            } else {
                // Si no se encuentra el tipo de usuario, mostrar un
mensaje de error
                JOptionPane.showMessageDialog(this, "El tipo de usuario
seleccionado no es válido");
            }
        } else {
            JOptionPane.showMessageDialog(this, "El password contiene
caracteres inválidos");
        }
    }
} catch (SQLException ex) {
    Logger.getLogger(MDI_Form.class.getName()).log(Level.SEVERE, null,
ex);
}
}
```

17. Ya que podemos tener problemas con el campo de edad, agregaremos una validación para evitar que se escriban caracteres diferentes de dígitos, para ello realizar lo siguiente:

- Ubicarse sobre el controltxtEdad y agregar el evento keyTyped.



- Modificar el evento tal y como se muestra a continuación

```
private void txtEdadKeyTyped(java.awt.event.KeyEvent evt) {
    // TODO add your handling code here:
    if(!Character.isDigit(evt.getKeyChar()))
    {

        Toolkit.getDefaultToolkit().beep();
          evt.consume();
    }
}
```

Se dará cuenta que ahora ya no deja escribir caracteres que no sean números.

NOTA: Realice las pruebas, esta práctica se utilizará en futuras prácticas.

ACTIVIDAD COMPLEMENTARIA

Investiga y realiza utilizando la base de datos proporcionada para el desarrollo de la guía, un login, el cual mediante los datos ingresados en el formulario 'Login', permita acceder al 'Panel de Administrador' realizado en la guía.

Login → Panel de Administrador

Debe estar debidamente validado ante cualquier error o excepción que se pueda disparar durante la ejecución del código y de ninguna manera se debe acceder al panel, sin antes pasar por el login.

BIBLIOGRAFÍA

- Oracle. (s. f.). *Java downloads*. Oracle.com. Recuperado el 4 de marzo de 2024, de https://www.oracle.com/java/technologies/downloads/
- Apache NetBeans. (s. f.). *Apache NetBeans releases*. Apache.org. Recuperado el 4 de marzo de 2024, de https://netbeans.apache.org/front/main/download/index.html
- Rodríguez, & Walter Sagástegui, A. (s. f.). *Configurar Java en Windows. Variables de entorno JAVA_HOME y PATH (CU00610B)*. Aprenderaprogramar.com. Recuperado el 4 de marzo de 2024, de https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=389:configurar-java-en-windows-variables-de-entorno-javahome-y-path-cu00610b&catid=68&Itemid=188